

Measurement

U_AT_29-09-2026

To calculate the temporal matching performance in strict 15-minute intervals, the database must aggregate the raw telemetry data using a specific set of mathematical formulas.

Here is the exact logic your MRV Engine will use to calculate production, consumption, and the resulting carbon deficit.

- **1. Total Consumption & Generation (15-Min Window)**

First, the system sums all individual smart meter readings (i) recorded within that specific 15-minute timeframe.

$$E_{consumed} = \sum_{i=1}^n C_i$$
$$E_{generated} = \sum_{i=1}^n G_i$$

(Where C_i represents each industrial consumption reading, G_i represents each renewable generation reading, and E is total Energy in kWh).

- **2. The Temporal Match (Actual Green Energy Used)**

A facility cannot claim to use more green energy than it actually consumed during that exact window. The matched energy is simply the lower value between what was generated and what was consumed.

$$E_{matched} = \min(E_{generated}, E_{consumed})$$

- **3. Temporal Match Score (%)**

This calculates the percentage of the facility's power that was genuinely green during that interval.

$$Score_{green} = \left(\frac{E_{matched}}{E_{consumed}} \right) \times 100$$

- **4. Carbon Deficit (%)** This formula powers the visual carbon deficit heatmap by pinpointing the exact intervals where clean generation fell short.

$$Deficit_{carbon} = 100 - Score_{green}$$

Concrete Example:

If an industrial grid consumes 500 kWh ($E_{consumed}$) between 2:00 PM and 2:15 PM, but the solar array only generates 300 kWh ($E_{generated}$) due to cloud cover:

- **Matched Energy** = $\min(300, 500) = 300$ kWh.
- **Green Score** = $(300/500) \times 100 = 60\%$.

- **Carbon Deficit** = $100 - 60 = 40\%$.

Your API will instantly flag this 40% deficit, proving that the facility was forced to rely on dirty grid backup for the remaining 200 kWh.

U_AT_07-10-2026

To accurately manage and calculate energy generation and consumption on a system, the architecture must bridge the gap between continuous physical physics (Power) and discrete digital data (Energy).

When building a scalable infrastructure-as-a-service platform, relying on simple addition is insufficient. Real-world IoT meters send noisy, discrete data packets that must be mathematically integrated, statistically cleaned, and programmatically managed.

Here is the exact mathematical, statistical, and computer science breakdown for calculating actual generated and consumed energy.

1. The Mathematics: Converting Power to Energy

The most common mistake in energy software is confusing **Power** (measured in Kilowatts - kW) with **Energy** (measured in Kilowatt-hours - kWh).

- **Power (P)** is the *rate* at which electricity is used at a specific microsecond.
- **Energy (E)** is the total amount of work done over a period of time.

Mathematically, Energy is the definite integral of Power over time:

$$E = \int_{t_0}^{t_1} P(t) dt$$

However, your FastAPI backend does not receive a continuous mathematical function; it receives discrete data points (e.g., a reading every 30 seconds). To calculate the actual energy (E) consumed or generated in a 15-minute window, you must apply **Numerical Integration**. The most robust method for this in software engineering is the **Trapezoidal Rule**.

If a smart meter sends Power readings $P_0, P_1, P_2 \dots P_n$ at timestamps $t_0, t_1, t_2 \dots t_n$, the Energy in kWh is calculated as:

$$E_{interval} = \sum_{i=1}^n \frac{P_{i-1} + P_i}{2} \times \left(\frac{t_i - t_{i-1}}{3600} \right)$$

(Note: The division by 3600 converts the time difference from seconds to hours, yielding kWh).

2. The Computer Science: State Management & Ingestion

To calculate these integrals in real-time without crashing the server, the system architecture must handle high-frequency data streams asynchronously.

- **Asynchronous Ingestion:** Your backend must use asynchronous workers to accept payloads from thousands of meters simultaneously. Instead of calculating the integral on the fly during the POST request, the API simply validates the JSON payload and pushes it to the database.
- **Time-Series Storage:** Standard relational databases choke on high-frequency temporal data. Utilizing a time-series optimized database like TimescaleDB allows you to store the raw P_i (kW) values and timestamps.
- **Continuous Aggregation:** You utilize the database's native functions (like TimescaleDB's continuous aggregates) to run the Trapezoidal integration formula natively in the database layer. This groups the raw seconds-level data into strict 15-minute buckets ($E_{generated}$ and $E_{consumed}$) automatically, keeping the API layer lightweight and fast.

3. The Statistics: Noise Reduction and Imputation

IoT sensors in industrial environments are notoriously unreliable. Packets drop, Wi-Fi fails, and sensors send anomalous spikes. If you blindly run the mathematical integral on raw data, your MRV audits will be invalid.

- **Handling Missing Data (Imputation):** If a meter fails to report for 2 minutes, your integration calculation will have a massive gap. You must apply statistical interpolation (such as Linear Interpolation) to fill the missing $P(t)$ values before integrating.

$$P_{missing} = P_{t-1} + (t_{missing} - t_{t-1}) \frac{P_{t+1} - P_{t-1}}{t_{t+1} - t_{t-1}}$$

- **Anomaly Detection:** Before accepting a data point, the system must apply a statistical filter. Using a rolling Z-score algorithm, if an incoming reading deviates by more than 3 standard deviations (σ) from the recent rolling mean (μ), it should be flagged as a hardware error rather than a genuine energy spike.

$$Z = \frac{P_i - \mu}{\sigma}$$

By structuring the pipeline this way, the backend ingests noisy kW readings, the statistical layer cleans them, and the mathematical layer integrates them into precise 15-minute kWh blocks ready for temporal matching and AI/ML time-series forecasting.

U_AT_07-10-2026

To calculate the exact metrics for your MRV platform, you must combine the raw data points—renewable generation, total consumption, grid import/export, and battery charging/discharging—into a unified mathematical model.

When presenting this architecture to potential enterprise clients or defending it in your final-year engineering submission, the core logic must prove that the system cannot be tricked by faulty IoT meters or fraudulent reporting.

Here are the four primary calculation methods your Python backend will execute for every 15-minute interval:

1. The Energy Balance Equation (The Anti-Fraud Check)

Before calculating any green scores, the system must verify "data completeness and abnormal readings." In physics, the total energy entering the system must equal the total energy leaving or being consumed.

$$E_{in} = E_{out}$$

$$(G_{ren} + Grid_{in} + Bat_{out}) = (C_{total} + Grid_{out} + Bat_{in} + Loss_{thermal})$$

(G_{ren} = Renewable Generation, Bat = Battery, $Loss_{thermal}$ = Acceptable minor hardware loss)

If a smart meter drops a packet or sends a corrupted reading, the left side of this equation will not match the right side. Your FastAPI engine should calculate this difference; if the variance exceeds a set threshold (e.g., 2%), the system automatically flags that 15-minute interval's data_quality_status as anomalous and halts the audit score generation.

2. Available Green Energy Calculation

A facility's clean energy during any interval comes from two sources: live generation (like solar) and stored clean energy discharging from the battery.

$$Green_{available} = G_{ren} + Bat_{out}$$

(Note: This assumes the battery was previously charged using only renewable energy. For a highly advanced startup product, you would track the "color" of the electrons in the battery, but for version 1.0, treating battery discharge as green is standard).

3. The True Temporal Match (Net Green Score)

A facility cannot claim to be 100% green if it generated 500 kWh of solar but exported 400 kWh to the grid while only consuming 100 kWh. The actual green energy utilized is the minimum value between what was available and what was consumed on-site.

$$Green_{matched} = \min(Green_{available}, C_{total})$$

To convert this into the compliance percentage for the UI dashboard:

$$Score_{green} = \left(\frac{Green_{matched}}{C_{total}} \right) \times 100$$

4. The Grid Deficit (Dirty Backup Auditing)

If the temporal match is less than 100%, the facility had to rely on the external utility grid. This calculation verifies exactly how much polluting energy was used.

$$Deficit_{kwh} = C_{total} - Green_{matched}$$

To cross-verify this, your system checks the $Deficit_{kwh}$ against the $Grid_{in}$ reading from the smart meter. The deficit should equal the grid import (unless the facility was simultaneously using the dirty grid to charge the battery, which would trigger a massive load-shifting alert).

By layering these four calculations in your analytics_engine module, your platform stops relying on blind trust. It uses thermodynamics to mathematically prove whether a facility is genuinely net-zero.